

Unit 2 - Entity Relationship Modelling (CO2)

Topics:

Components of an E-R Model: Entity, Entity Sets, Attributes and keys - Super Key, Primary Key, Candidate Key, Alternate Key, Relationship Types, Composite entities, Weak entity

Relational Database design by ER to Relational Mapping

Examples for ER Diagram: College Management System, Railway Reservation System, Hospital Management System, Library Management System, e-Business Management System (can take any eBusiness example like online shopping, online food ordering etc....)

Entities and Attributes

▶ Entity:

- ▶ A thing or an object.
- ▶ It has set of properties
- ▶ Values for some set of properties, it can uniquely identify it
- ▶ Entities are specific objects or things in the mini-world that are represented in the database.
- ▶ For example the EMPLOYEE John Smith, the Research DEPARTMENT, the ProductX PROJECT

▶ Attributes are properties used to describe an entity.

- ▶ For example an EMPLOYEE entity may have the attributes Name, SSN, Address, Sex, BirthDate

▶ A specific entity will have a value for each of its attributes.

- ▶ For example a specific employee entity may have Name='John Smith', SSN='123456789', Address ='731, Fondren, Houston, TX', Sex='M', BirthDate='09-JAN-55'

▶ Each attribute has a value set (or data type) associated with it - e.g. integer, string, subrange, enumerated type, ...

Type of Attributes

- ▶ Simple Attribute
 - ▶ Each entity has a single atomic value for the attribute.
 - ▶ For example: SSN or Gender.
- ▶ Composite Attribute
 - ▶ An attribute where the values of that attribute can be further subdivided into meaningful sub-parts."
 - ▶ For Example: Emp_Name - may be stored as first name, last name, middle initial
 - ▶ Address(Apt#, House#, Street, City, State, ZipCode, Country)
- ▶ Multivalued Attribute
 - ▶ An attribute that can have more than one value associated with the key of the entity.
 - ▶ For Example: Degree of a person, Hobby of a person, Bank branch Location, Color of a car etc.
- ▶ Derived Attribute
 - ▶ An attribute whose value is calculated (derived) from other attributes.
 - ▶ The derived attribute need not be physically stored within the database; instead, it can be derived by using an algorithm.
 - ▶ For Example: Age field (can be derived from Birth_date field)
- ▶ Key Attribute
 - ▶ Uniquely identifies each entity in an entity set.
 - ▶ Example: Roll_No for Student, Emp_ID for Employee.

▶ Composite and Multi-Valued Attributes (Nested Attributes)

- ▶ In general, **composite** and **multi-valued** attributes can be **nested arbitrarily** to any number of levels, though this is **rare** in practice.
- ▶ A **composite attribute** is one that can be divided into smaller sub-parts.
- ▶ A **multi-valued attribute** can hold **multiple values** for a single entity.
- ▶ When both properties exist together, the attribute is called a **composite multi-valued attribute**.
- ▶ **Example:** student entity, PreviousDegrees attribute
 - ▶ PreviousDegrees is **multi-valued** because a student can have more than one previous degree.
 - ▶ Each PreviousDegree has sub-parts, so it is **composite**: {PreviousDegrees (College, Year, Degree, Field)}

Entity Type and Entity Sets

- ▶ A database usually contains groups of entities that are similar
 - ▶ For Ex. Company Employees
- ▶ Each entity shares the same attribute but each entity has its own values for each attribute
- ▶ **Entity Type:** A collection of entities that have the same attributes
- ▶ **Entity Set:** The collection (or group) of all entities of a particular entity type stored in the database.
- ▶ Entity set is usually referred by using the same name as the entity type.
 - ▶ For Ex.: EMPLOYEE refers to both type of entity as well as the collection of all *employee* entities in the Database.

Relationship

- ▶ A relationship relates two or more distinct entities with a specific meaning.
 - ▶ Eg. Employee WORKS FOR Department -----"Work For" is the relationship
 - ▶ Student ENROLLS Course -----"Enrolls" is the relationship
- ▶ A relationship type R among n entity types E1, E2, ..., En defines a set of associations—or a relationship set—among entities from these entity types.
- ▶ As for the case of entity types and entity sets, a relationship type and its corresponding relationship set are customarily referred to by the same name, R.

Relationship instances

- ▶ In the mini-world represented by figure below employees e1, e3 and e6 work for department d1, e2 and e4 works for d2 and e5 and e7 work for d3.

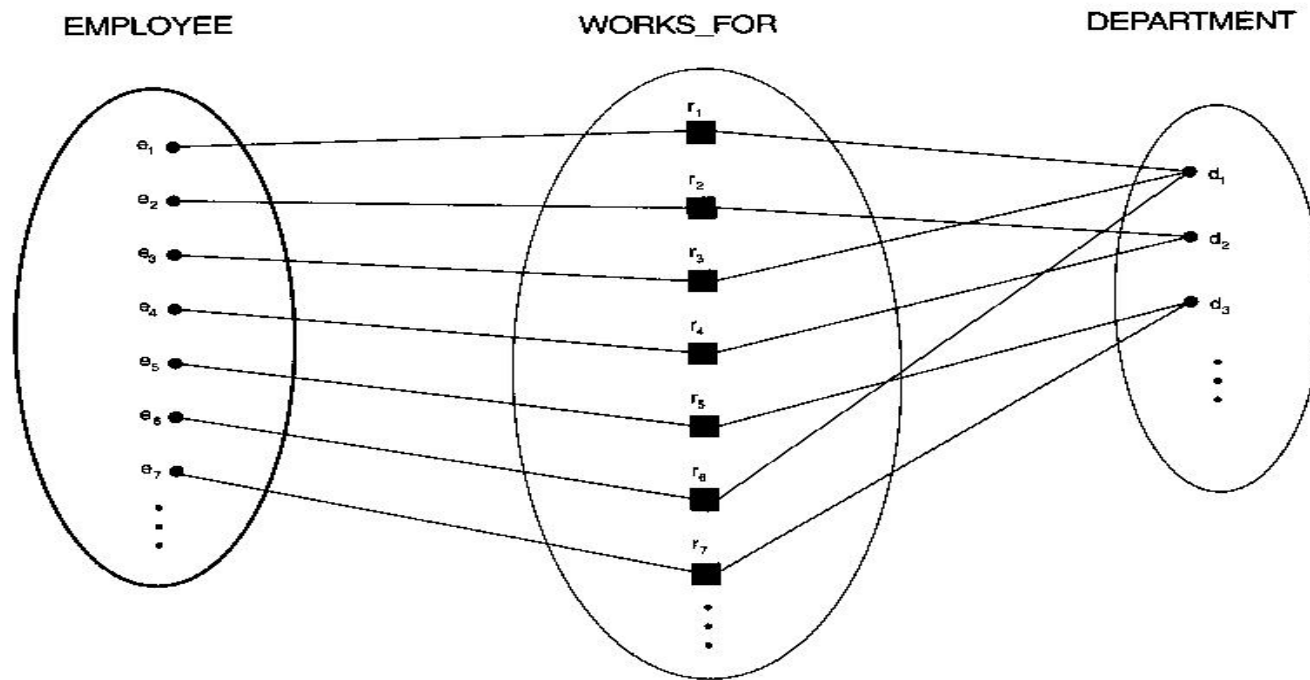
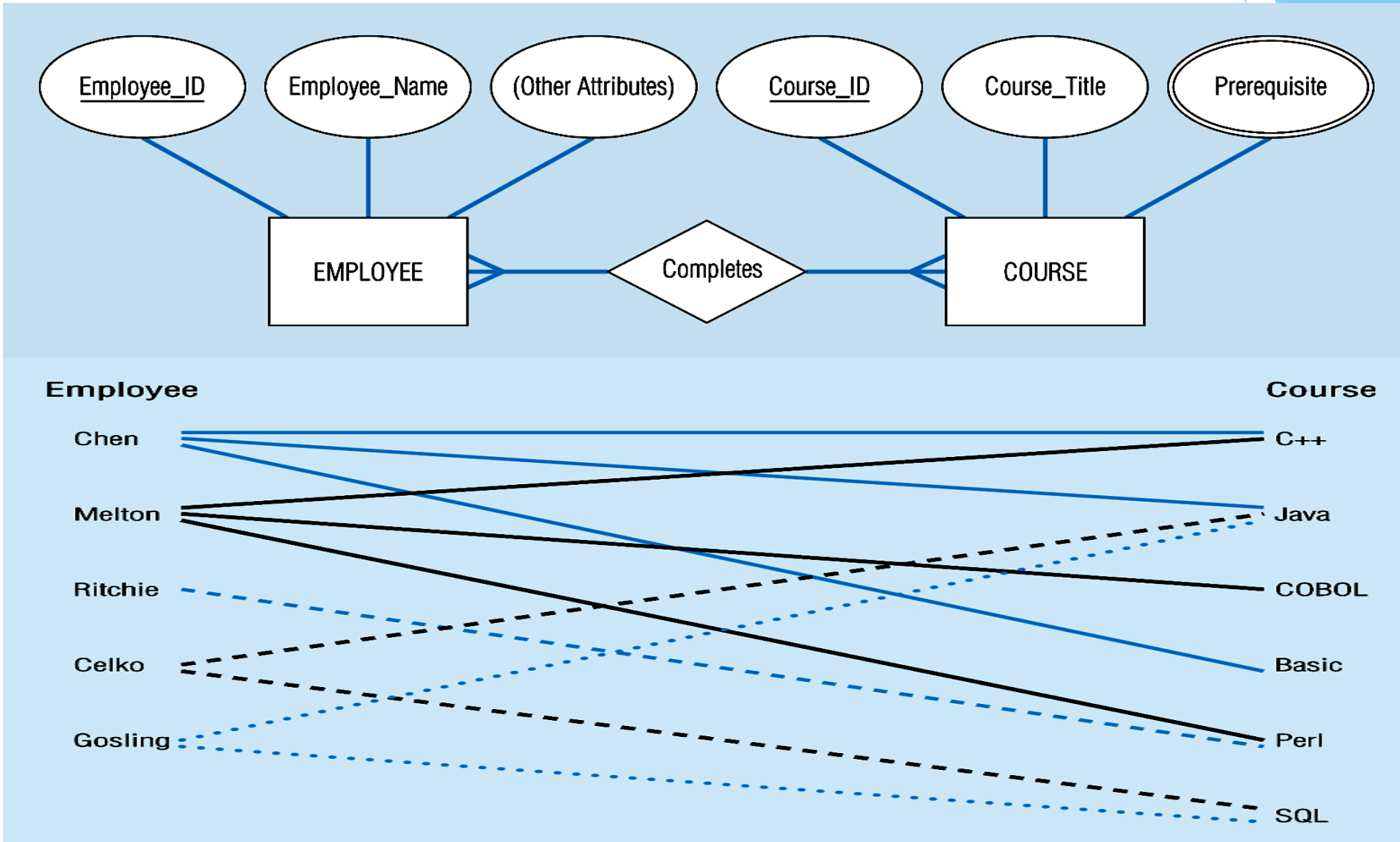


Figure 3.9 Some instances of the **WORKS_FOR** relationship between **EMPLOYEE** and **DEPARTMENT**.

Relationship types and instances



Relationship type vs. relationship set

- ▶ Previous figures displayed the relationship sets
- ▶ Each instance in the set relates individual participating entities - one from each participating entity type
- ▶ In ER diagrams, we represent the *relationship type* as follows:
 - ▶ Diamond-shaped box is used to display a relationship type
 - ▶ Connected to the participating entity types via straight lines

Relationship type vs. relationship set

- ▶ Relationship Type:
 - ▶ Defines the **association (link)** between two or more **entity types**.
 - ▶ Is the schema description of a relationship
 - ▶ Identifies the relationship name and the participating entity types
 - ▶ Also identifies certain relationship constraints
- ▶ Relationship Set:
 - ▶ A **collection of all relationships (instances)** of a particular relationship type in the database.
 - ▶ It contains **actual data** showing which entities are related.
 - ▶ The current set of relationship instances represented in the database
 - ▶ The current *state* of a relationship type

Cardinality and Participation Constraints

- ▶ In general, cardinality tells you "How many".
- ▶ Cardinality can be:
 - ▶ One to one (1:1): every time one entity occurs, there is exactly one occurrence of another entity.
 - ▶ One to many (1:m): every time one entity occurs, there are multiple occurrences of another entity.
 - ▶ Many to many (m:m): for each occurrence of an entity, there can be one or many occurrences of another—an vice versa.
- ▶ Participation Constraints:
 - ▶ Total Participation
 - ▶ Partial Participation

Example for 1:1 Relationship Type

- ▶ In an **Entity-Relationship (ER) Model**, a **1:1 (one-to-one) relationship** means:
- ▶ **One instance of Entity A is associated with only one instance of Entity B, and vice versa.**

- ◆ **Example 1: Person - Passport**

Person → Each person has **one passport**.

Passport → Each passport belongs to **one person** only.

- ◆ **Example 2: Husband - Wife**

Husband → one wife only.

Wife → one husband only.

📌 **Key Point:** In ER diagrams, the **1:1 relationship** is shown by connecting two entities with a line and placing **1** on both ends.

Example for 1:M Relationship Type

- ▶ In an **Entity-Relationship (ER) Model**, a **1:M (one-to-many) relationship** means:
- ▶ **One instance** of Entity A can be associated with **many instances** of Entity B.
- ▶ But each instance of Entity B is related to **only one instance** of Entity A.

◆ Example 1: Teacher - Students

Teacher → One teacher can teach **many students**.

Student → Each student is taught by **one teacher** (in this relationship).

◆ Example 2: Department - Employees

Department → One department can have **many employees**.

Employee → Each employee belongs to **only one department**.

◆ Example 3: Mother - Children

Mother → One mother can have **many children**.

Child → Each child has **one biological mother**

✚ **Key point:** "1" is placed on the side where only one instance exists, and "M" (many) is placed on the side where multiple instances exist.

Example for N:M Relationship Type

- ▶ In an **Entity-Relationship (ER) Model**, a **many-to-many (M:N)** relationship means:
- ▶ One instance of **Entity A** can be associated with **many instances** of **Entity B**,
- ▶ and one instance of **Entity B** can also be associated with **many instances** of **Entity A**.

◆ Example 1: Students - Courses

Student → One student can enroll in **many courses**.

Course → One course can have **many students** enrolled.

◆ Example 2: Authors - Books

Author → One author can write **many books**.

Book → One book can be written by **many authors**

◆ Example 3: Doctors - Patients

Doctor → A doctor can treat **many patients**.

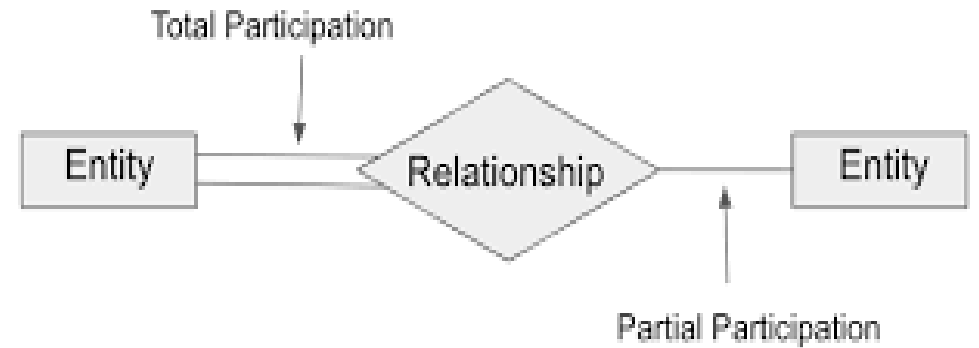
Patient → A patient can be treated by **many doctors**.

✚ **Key point:** In practice, an **M:N** relationship is usually broken into two **1:M** relationships using a **bridge/associative entity** (like *Enrollment* between Student & Course).

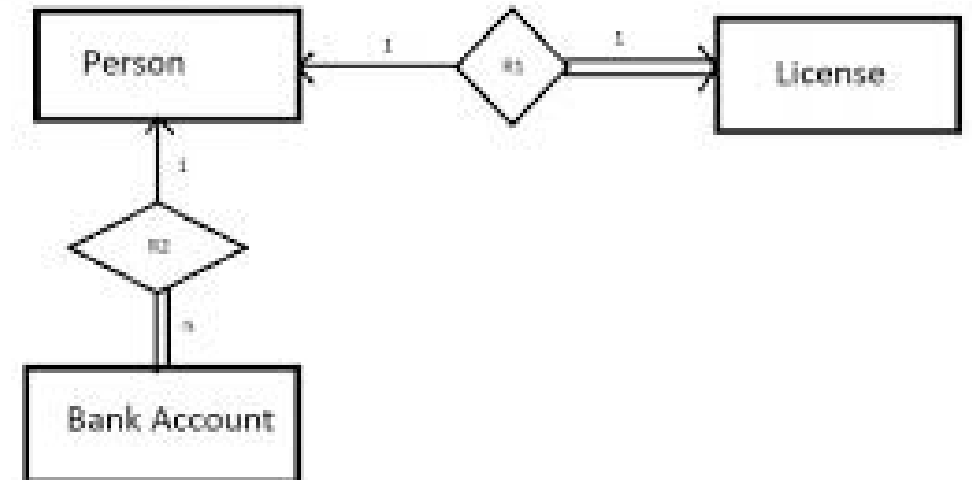
Participation Constraint

Example

- ◆ **Total Participation** – each student must enroll in a course
- ◆ **Partial Participation** – If some courses are not enrolled by any students, the participation of the course will be partial.



Participation in Enrolled relationship set: **Partial** Course
Total Student



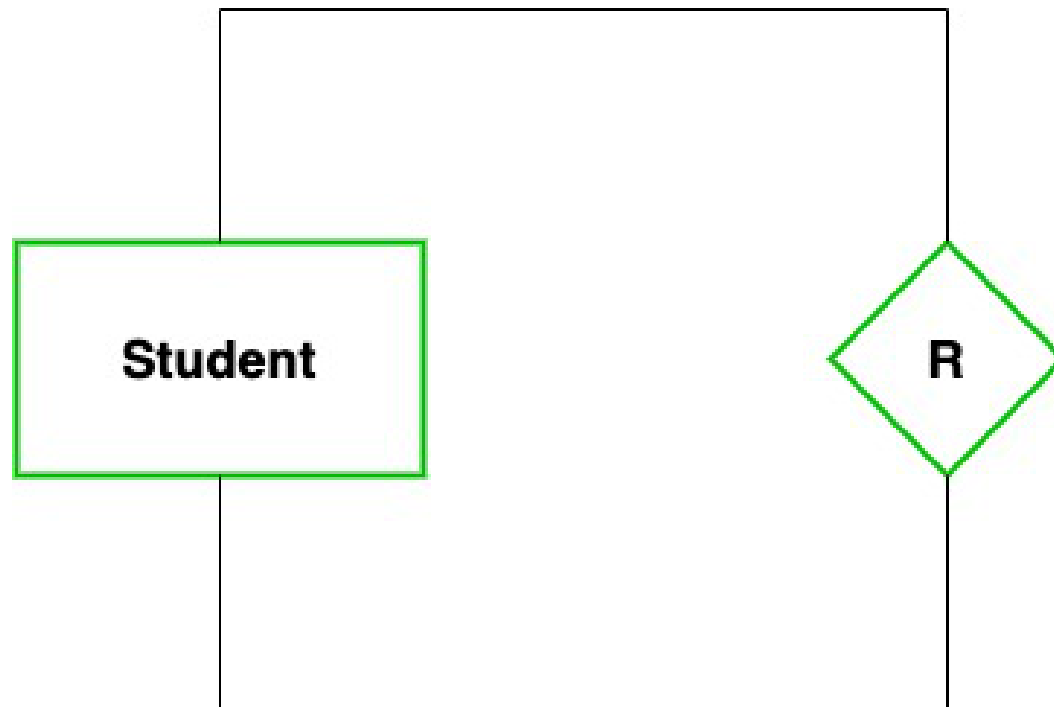
Degree of a Relationship

- ▶ Based on the number of linked entity types, we have 4 types of degrees of relationships.
 - ▶ Unary
 - ▶ Binary
 - ▶ Ternary
 - ▶ N-ary

Unary Relationship

- ▶ In this type of relationship, both the associating entity type are the same. So, we can say that unary relationships exist when both entity types are the same and we call them the degree of relationship is 1. Or in other words, in a relation only one entity set is participating then such type of relationship is known as a unary relationship.
- ▶ Example: In a particular class, we have many students, there are monitors too. So, here class monitors are also students. Thus, we can say that only students are participating here. So the degree of such type of relationship is 1.

Example - Unary Relationship

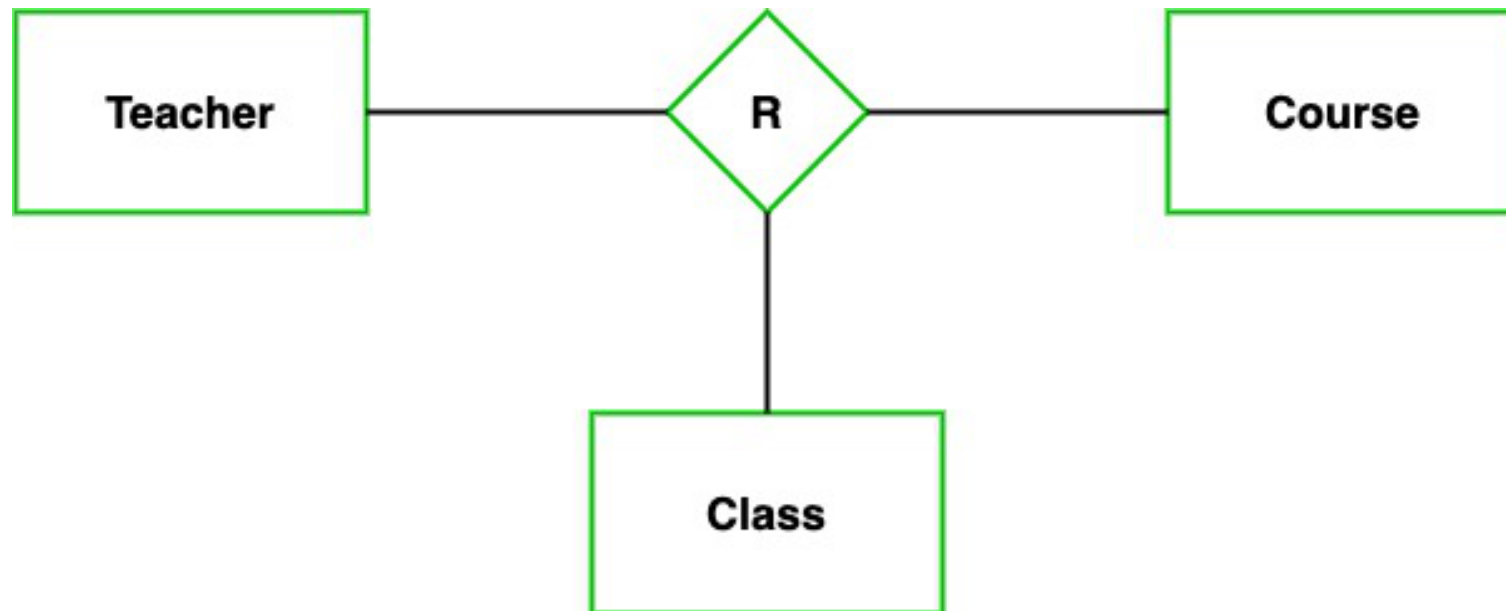


Binary Relationship

- ▶ In a Binary relationship, there are two types of entity associates.
- ▶ Binary relationship exists when there are two types of entity and we call them a degree of relationship is 2.
- ▶ Or in other words, in a relation when two entity sets are participating then such type of relationship is known as a binary relationship. This is the most used relationship and one can easily be converted into a relational table.

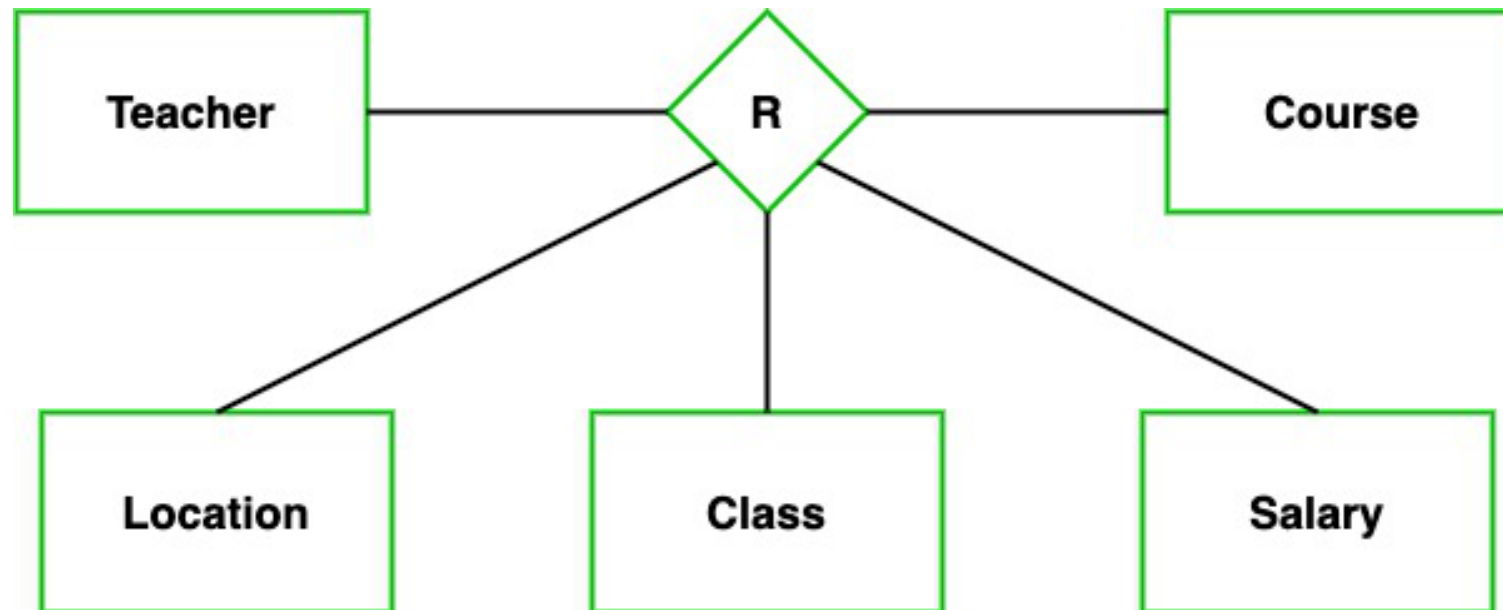
Ternary Relationship

- ▶ In the Ternary relationship, there are three types of entity associates.
- ▶ Ternary relationship exists when there are three types of entity and we call them a degree of relationship is 3



N-ary Relationship

- ▶ In the N-ary relationship, there are n types of entity that associates.
- ▶ N-ary relationship exists when there are n types of entities.



Company Management System

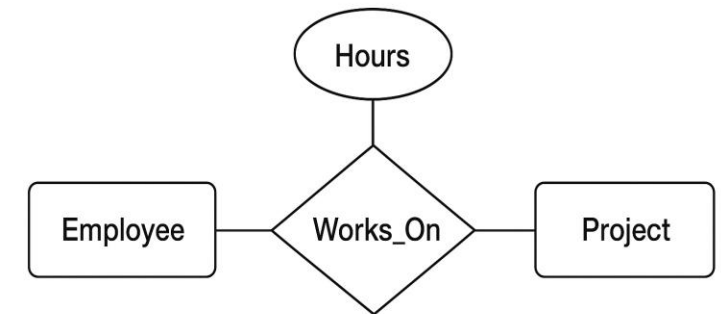
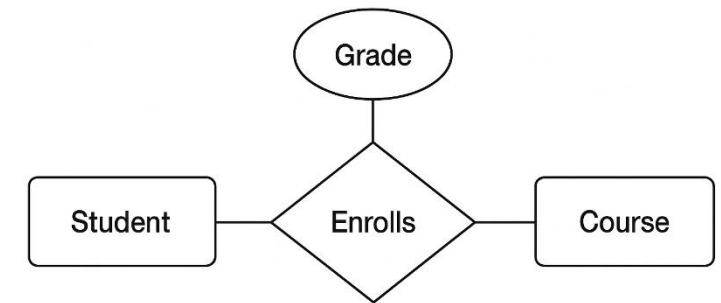
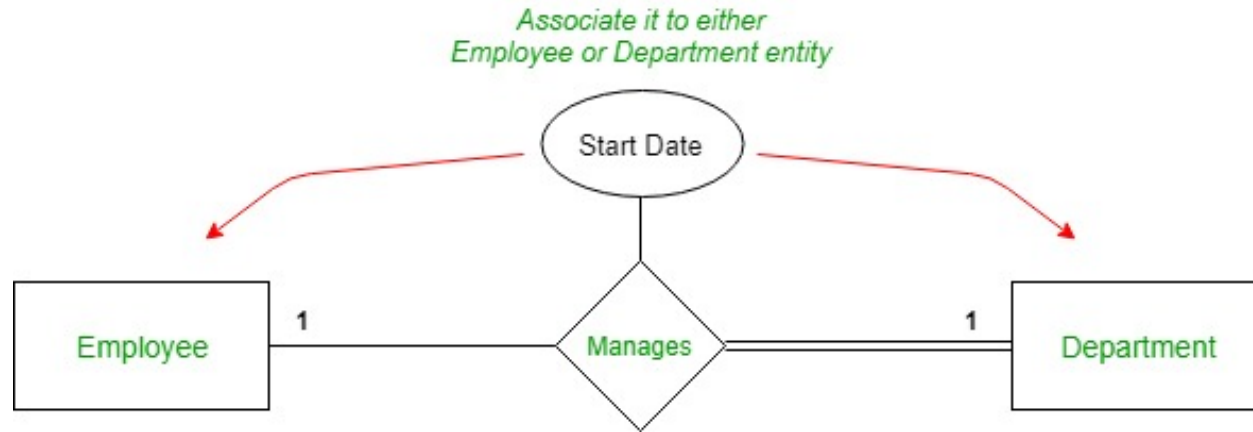
- ▶ Organization has multiple departments
- ▶ Employees work for the department and manages the department
- ▶ Per department there is an supervisor (an employee)
- ▶ Department executes different projects
- ▶ Employees work on those projects on hourly basis
- ▶ Organization manages the information of the Employee's dependents also.

Weak Entity Types

- ▶ An entity that does not have a key attribute
- ▶ A weak entity must participate in an identifying relationship type with an owner or identifying entity type
- ▶ Entities are identified by the combination of:
 - ▶ A partial key of the weak entity type
 - ▶ The particular entity they are related to in the identifying entity type
- ▶ **Example: 1**
 - ▶ A DEPENDENT entity is identified by the dependent's first name, *and* the specific EMPLOYEE with whom the dependent is related
 - ▶ Name of DEPENDENT is the *partial key*
 - ▶ DEPENDENT is a *weak entity type*
 - ▶ EMPLOYEE is its identifying entity type via the identifying relationship type DEPENDENT_OF
- ▶ **Example: 2**
 - ▶ ORDER (Strong Entity) and ORDERITEM (Weak Entity).
 - ▶ An OrderItem cannot exist without an Order.
 - ▶ Partial key: ItemNumber within an order.
 - ▶ Weak relationship: "Contains" between Order and OrderItem.

Relationship Type Attribute

- ▶ Relationship attribute is a specific characteristic or property of that relationship itself, providing more detail about how the entities are connected



Doctor ----- Treats(Date_of_Treatment) ----- Patient

Employee ----- Works_On(Hours) ----- Project

Example

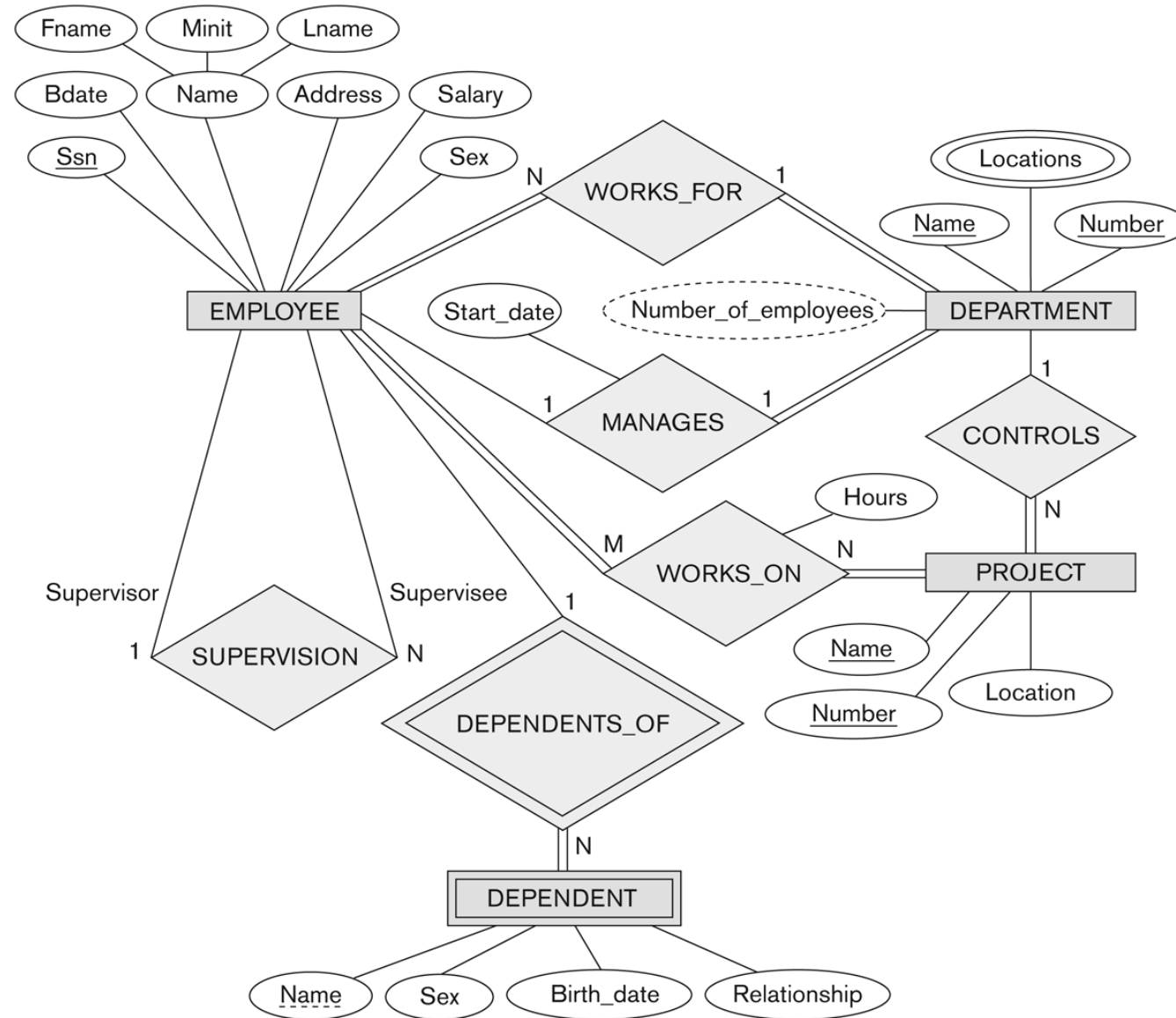


Figure 3.2

An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter.

University Management System

- ▶ Different College / institute are there under the university
- ▶ Each College has different departments that manages information related to courses
- ▶ Students are enrolled in a course
- ▶ Courses offers different subjects
- ▶ Faculties teaches subjects in different courses
- ▶ One faculty in each department is the head of that department
- ▶ One faculty is the director of the college / institute.
- ▶ Course has exam, students appear for the exam and get grades for the same

Draw an ER Diagram for the Library Management System.

- ▶ In a Library different types of books are purchased for the students to refer.
- ▶ Books are categorized by different authors and publishers.
- ▶ Student registers in the library and can issue books.
- ▶ Librarian manages student details.

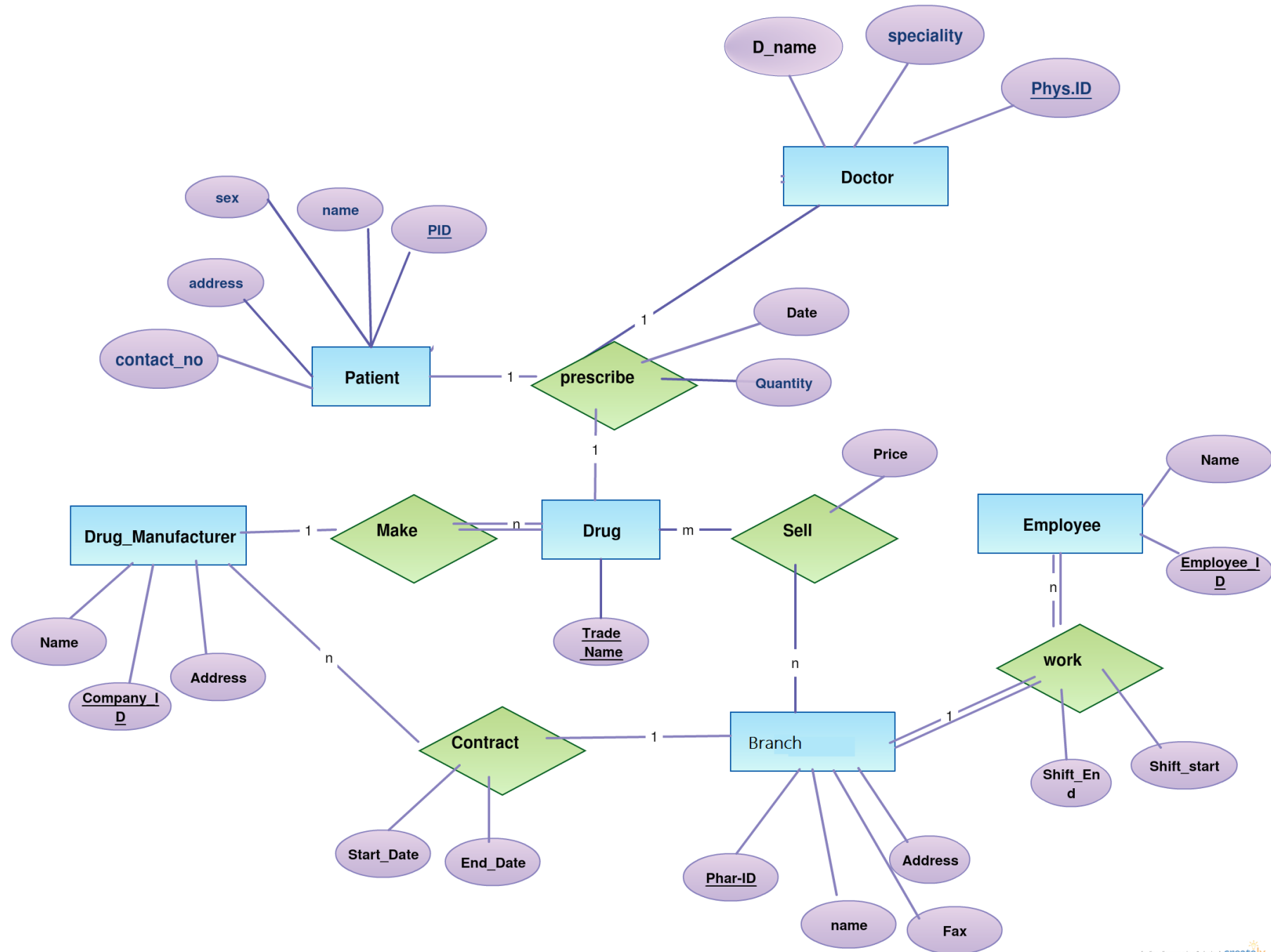
Project Management System of an IT Company

- ▶ A company handles different projects from clients
- ▶ Manages client information
- ▶ Project contains different task and duration
- ▶ Employee has to maintain the daywise timesheet for the task completed

Pharmacy Store Management

- ▶ Store has many branches, pharmacist as an employee in it
- ▶ Drug manufacturing company supplies drugs at different branches
- ▶ Patient come with a doctor's prescription
- ▶ Pharmacist sell drugs
- ▶ The store manages information relate to the doctor, the patient as a customer, the manufacturer and the pharmacist as an employee - branch wise

E-R Diagram for Pharmacy Store Information



ER Model to Relational Model

- ▶ ER diagrams mainly comprise of – Entity and its attributes Relationship, which is association among entities.

STEP 1:

▶ Mapping Entity Process *Algorithm*

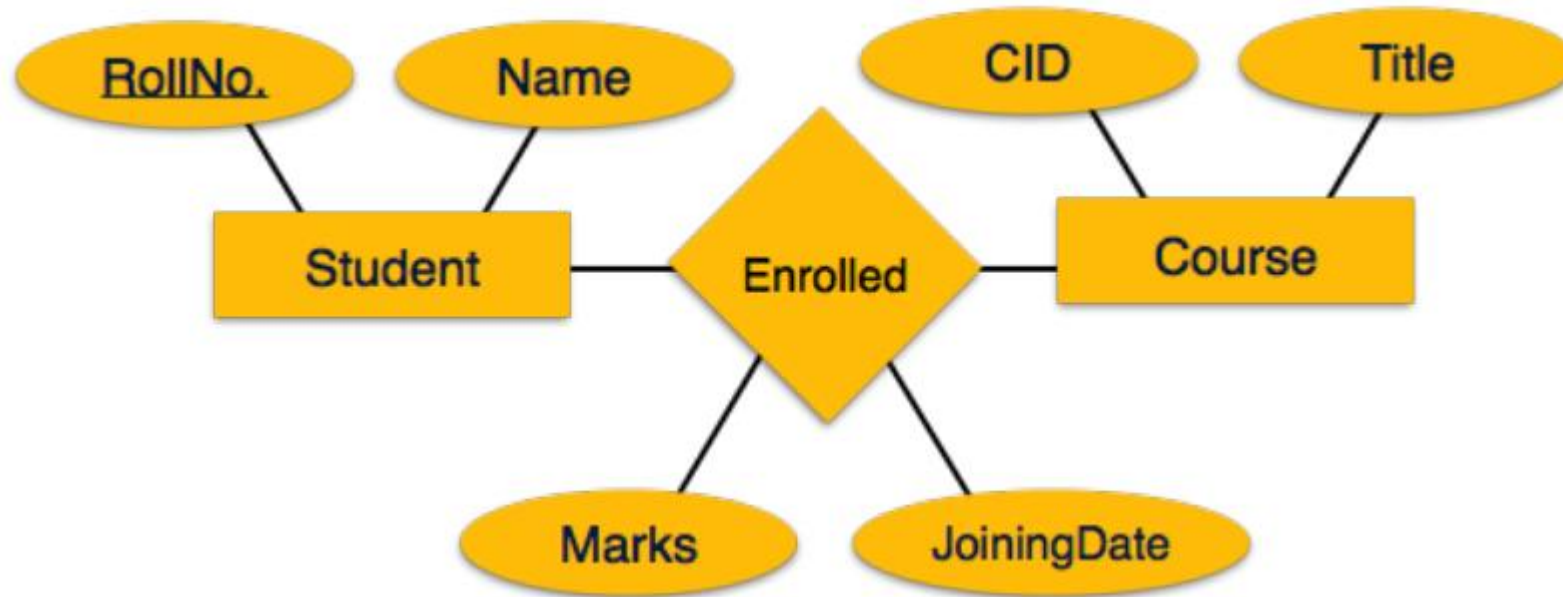
- ▶ Create table for each entity.
- ▶ Entity's attributes should become fields of tables with their respective data types.
- ▶ Declare primary key.
- ▶ Example: We create the relations EMPLOYEE, DEPARTMENT, and PROJECT in the relational schema corresponding to the regular entities in the ER diagram.
 - ▶ SSN, DNUMBER, and PNUMBER are the primary keys for the relations EMPLOYEE, DEPARTMENT, and PROJECT as shown.

ER Model to Relational Model

STEP 2:

▶ Mapping Relationship Process *Algorithm*

- ▶ Create table for a relationship if it is a **n:m (many to many)relationship**.
 - ▶ Add the primary keys of all participating Entities as fields of table with their respective data types.
 - ▶ If relationship has any attribute, add each attribute as field of table.
 - ▶ Declare a primary key composing all the primary keys of participating entities.
 - ▶ Declare all foreign key constraints.
-
- ▶ Example: The M:N relationship type WORKS_ON from the ER diagram is mapped by creating a relation WORKS_ON in the relational database schema.
 - ▶ The primary keys of the PROJECT and EMPLOYEE relations are included as foreign keys in WORKS_ON and renamed PNO and ESSN, respectively.
 - ▶ Attribute HOURS in WORKS_ON represents the HOURS attribute of the relation type. The primary key of the WORKS_ON relation is the combination of the foreign key attributes {ESSN, PNO}.



Create a table for Enrolled with fields:
(Roll_No, CID, Marks, JoiningDate)

Mapping Relationship type

► Mapping of Binary 1:1 Relation Types

- For each binary 1:1 relationship type R in the ER schema, identify the relations S and T that correspond to the entity types participating in R.
- There are three possible approaches:
 1. **Foreign Key approach:** Choose one of the relations-say S-and include a foreign key in S the primary key of T. It is better to choose an entity type with total participation in R in the role of S.
 - Example: 1:1 relation MANAGES is mapped by choosing the participating entity type DEPARTMENT to serve in the role of S, because its participation in the MANAGES relationship type is total.
 2. **Merged relation option:** An alternate mapping of a 1:1 relationship type is possible by merging the two entity types and the relationship into a single relation. This may be appropriate when both participations are total.
 3. **Cross-reference or relationship relation option:** The third alternative is to set up a third relation R for the purpose of cross-referencing the primary keys of the two relations S and T representing the entity types.

Mapping Relationship type

▶ Mapping of 1 : N Relationship

- ▶ For each regular binary 1:N relationship type R, identify the relation S that represent the participating entity type at the N-side of the relationship type.
- ▶ Include as foreign key in S the primary key of the relation T that represents the other entity type participating in R.
- ▶ Include any simple attributes of the 1:N relation type as attributes of S.
 - ▶ Example: 1:N relationship types WORKS_FOR, CONTROLS, and SUPERVISION in the figure.
 - ▶ For WORKS_FOR we include the primary key DNUMBER of the DEPARTMENT relation as foreign key in the EMPLOYEE relation and call it DNO.

Mapping Relationship type

► Mapping of N-ary Relationship.

- For each n-ary relationship type R, where $n > 2$, create a new relationship S to represent R.
- Include as foreign key attributes in S the primary keys of the relations that represent the participating entity types.
- Also include any simple attributes of the n-ary relationship type (or simple components of composite attributes) as attributes of S.

► Example: The relationship type SUPPY in the ER on the next slide.

- This can be mapped to the relation SUPPLY shown in the relational schema, whose primary key is the combination of the three foreign keys {SNAME, PARTNO, PROJNAME}

FIGURE 4.11

Ternary relationship types. (a) The SUPPLY relationship.

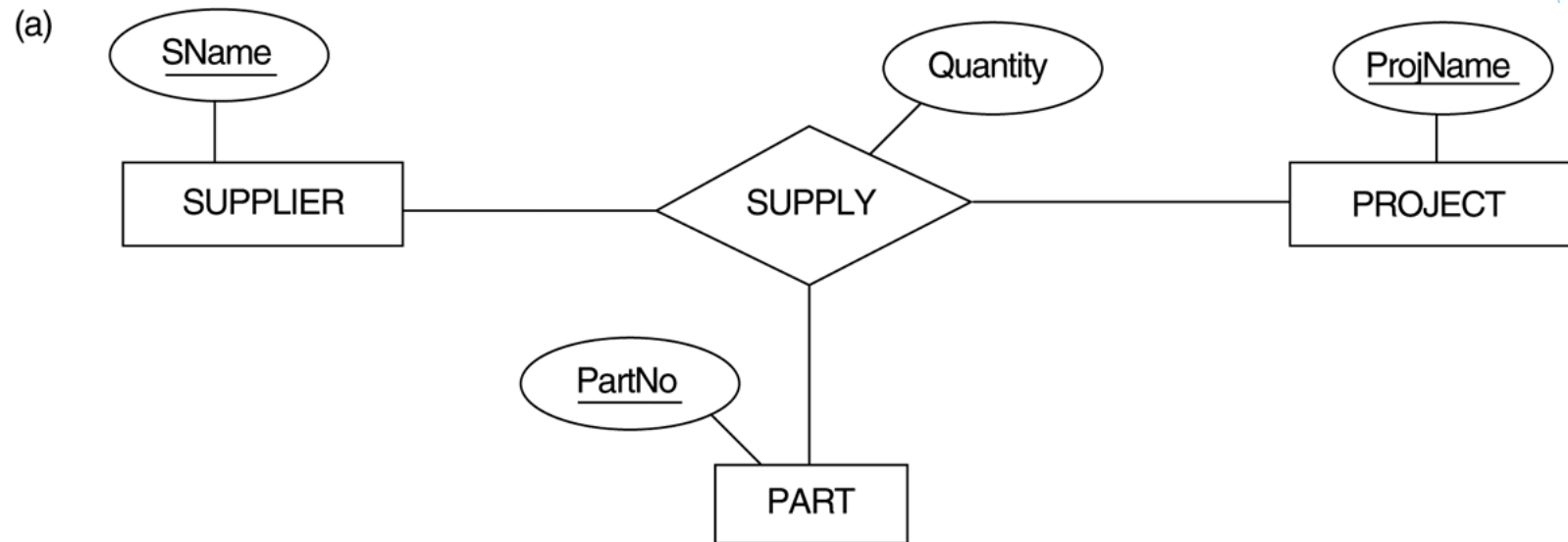


FIGURE 7.3

Mapping the *n*-ary relationship type SUPPLY from Figure 4.11a.

SUPPLIER

<u>SNAME</u>	...
--------------	-----

PROJECT

<u>PROJNAME</u>	...
-----------------	-----

PART

<u>PARTNO</u>	...
---------------	-----

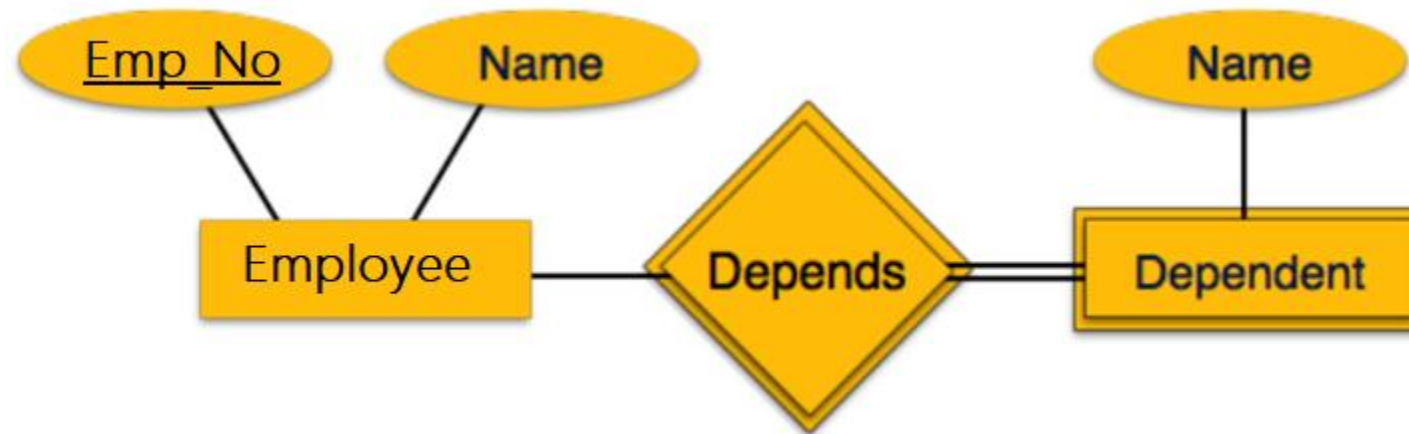
SUPPLY

<u>SNAME</u>	PROJNAME	<u>PARTNO</u>	QUANTITY
--------------	----------	---------------	----------

ER Model to Relational Model (Cont...)

STEP 3:

- ▶ Mapping Weak Entity Sets Process *Algorithm*
 - ▶ Create table for weak entity set.
 - ▶ Add all its attributes to table as field.
 - ▶ Add the primary key of identifying entity set.
 - ▶ Declare all foreign key constraints.



ER-to-Relational Mapping Algorithm (contd.)

▶ Step 4: Mapping of Multivalued attributes.

- ▶ For each multivalued attribute A, create a new relation R.
- ▶ This relation R will include an attribute corresponding to A, plus the primary key attribute K-as a foreign key in R-of the relation that represents the entity type of relationship type that has A as an attribute.
- ▶ The primary key of R is the combination of A and K. If the multivalued attribute is composite, we include its simple components.

▶ Example: The relation DEPT_LOCATIONS is created.

- ▶ The attribute DLOCATION represents the multivalued attribute LOCATIONS of DEPARTMENT, while DNUMBER-as foreign key-represents the primary key of the DEPARTMENT relation.
- ▶ The primary key of R is the combination of {DNUMBER, DLOCATION}.

Example

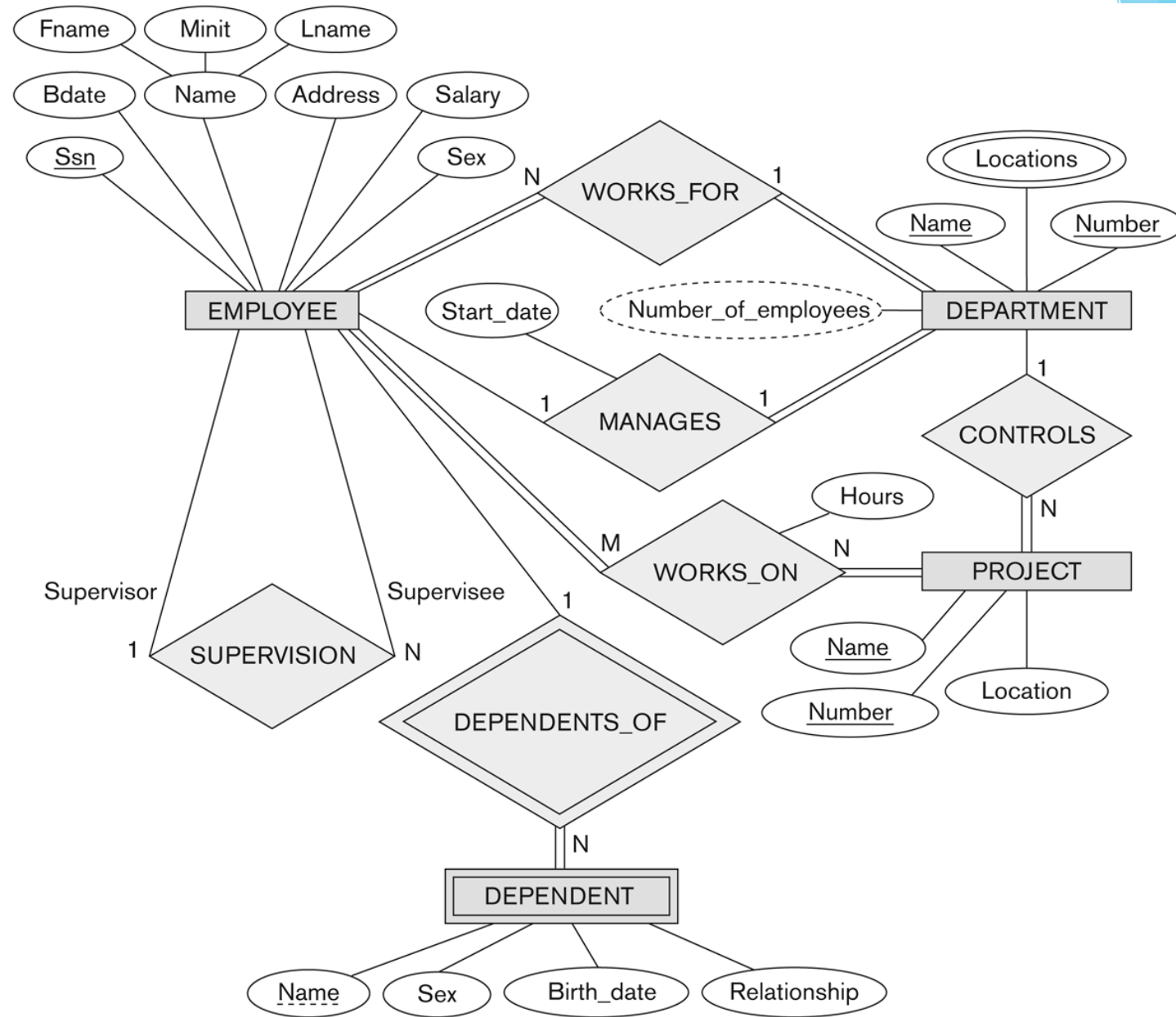


Figure 3.2

An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter.

FIGURE 7.2
Result of mapping the COMPANY ER schema into a relational schema.

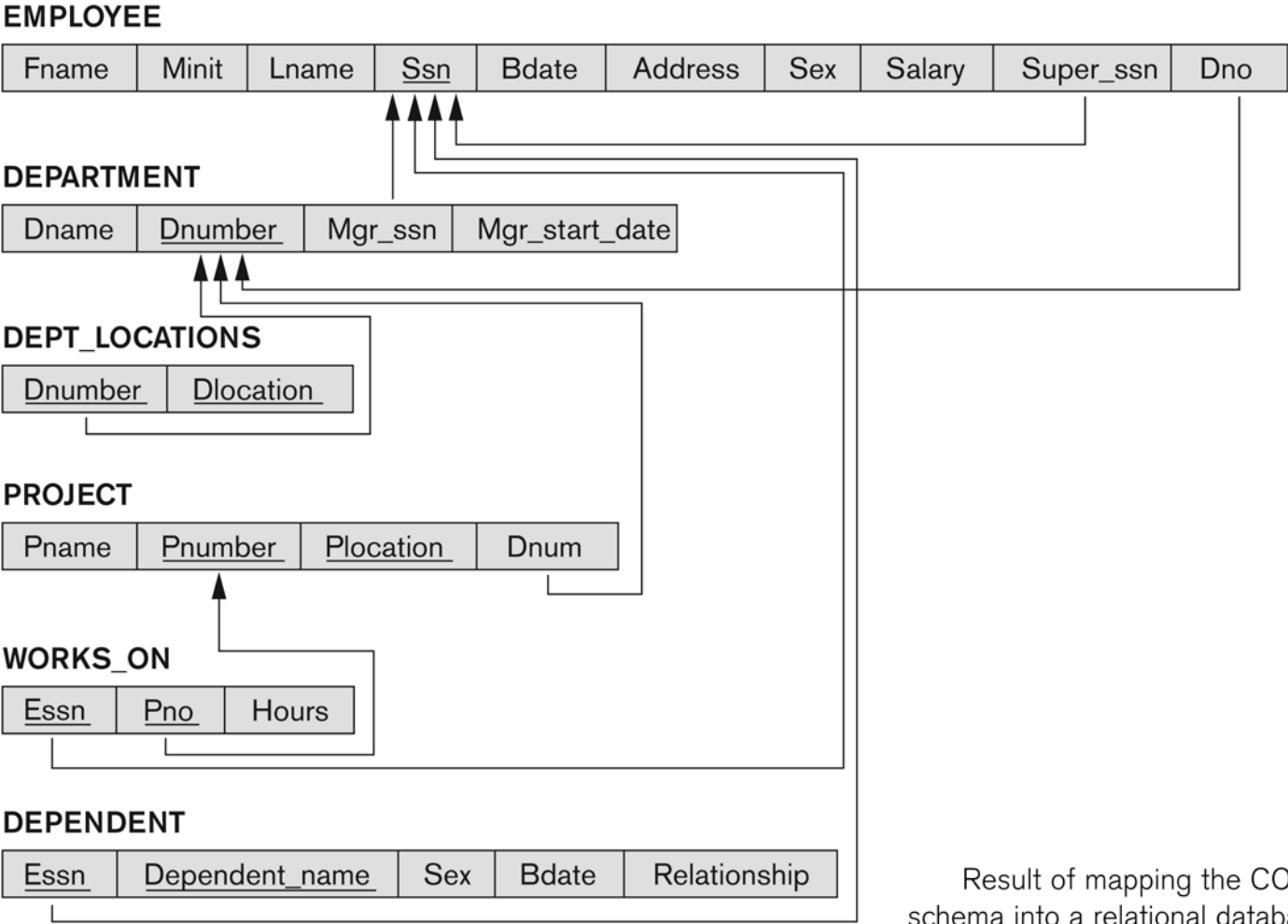


Figure 7.2
Result of mapping the COMPANY ER
schema into a relational database schema.

Example

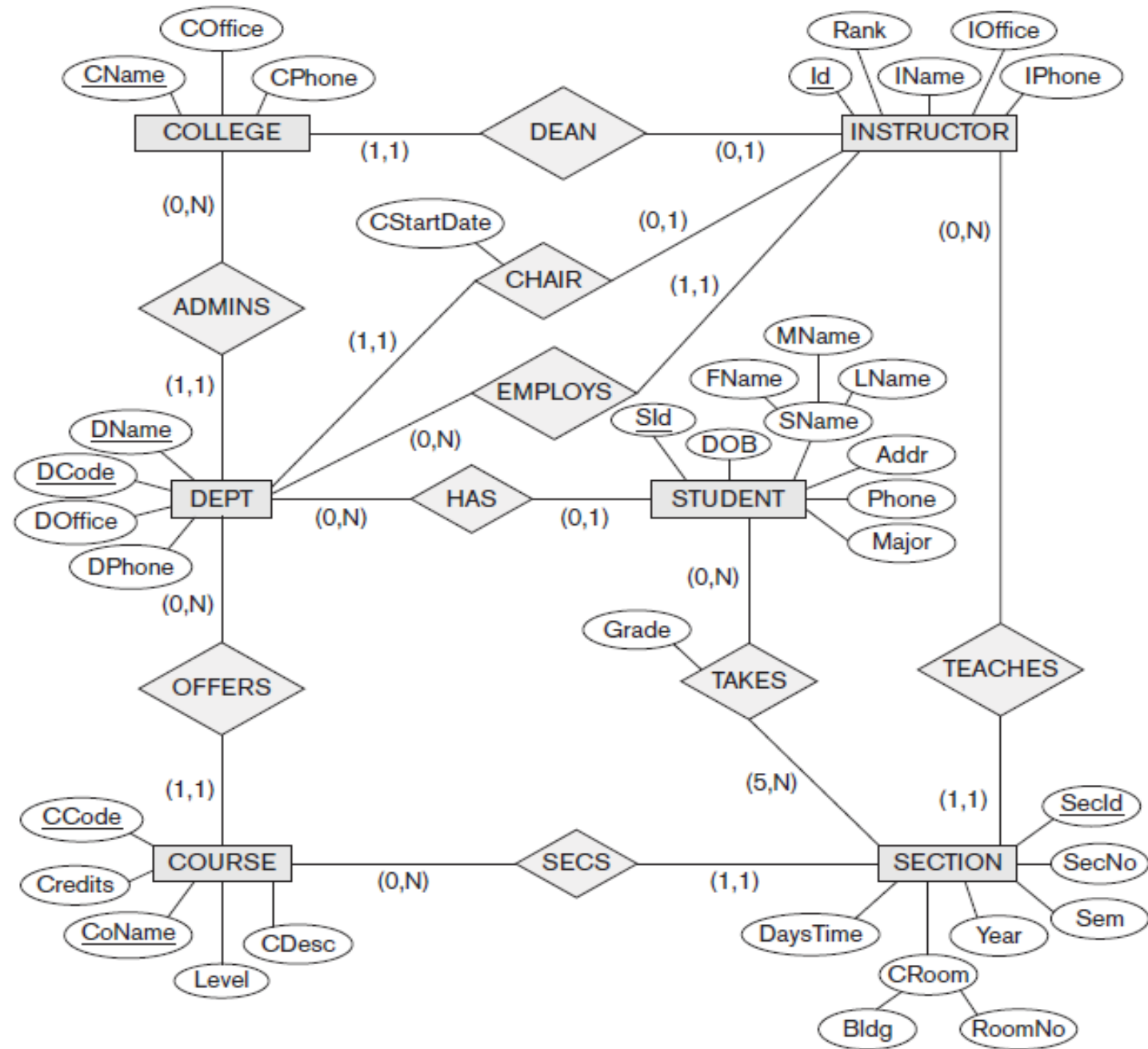


Figure 3.20
An ER diagram for a UNIVERSITY database schema.

Mapping Exercise

Exercise 7.4.

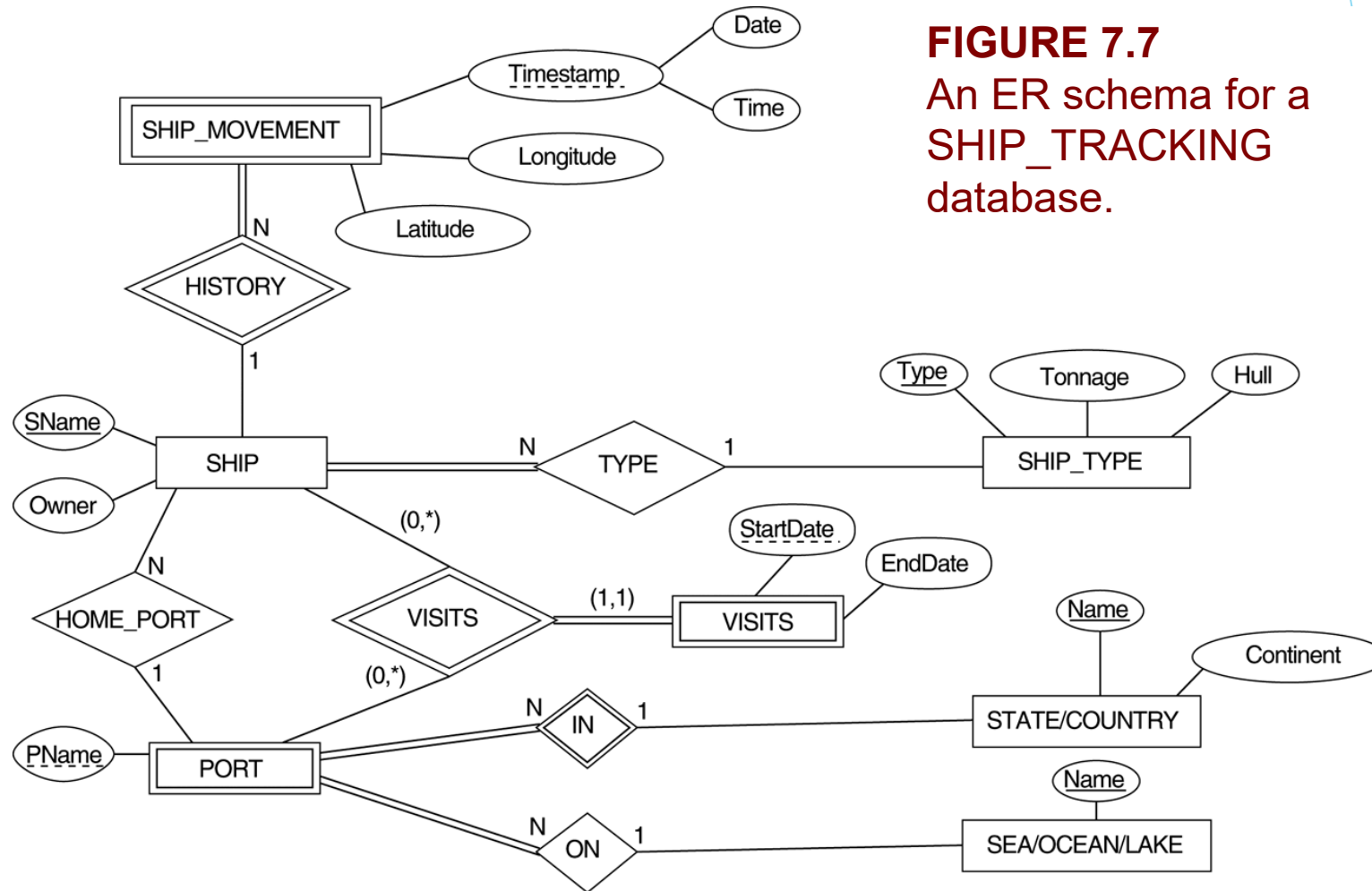


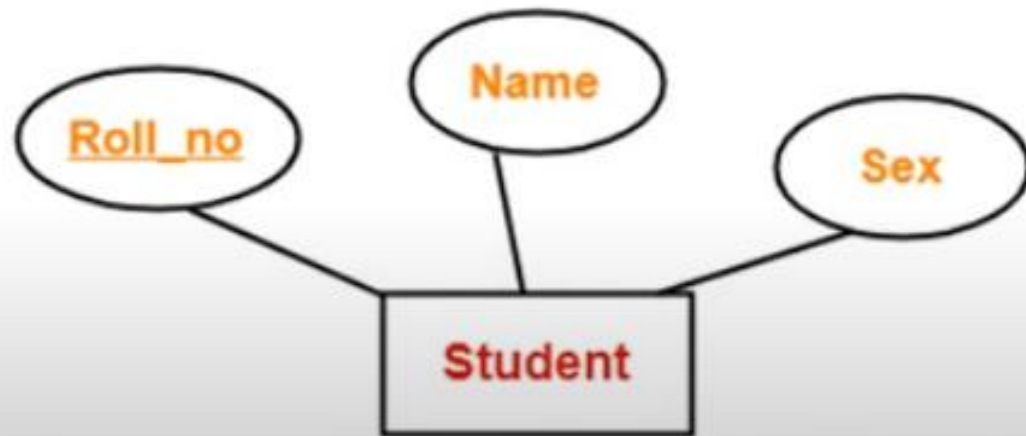
FIGURE 7.7

An ER schema for a SHIP_TRACKING database.

Mapping ER to Table

➤ Rule 1: Strong Entity set with Simple attributes

- Attributes of the table will be the attributes of the entity set.
- The primary key of the table will be the key attribute of the entity set.

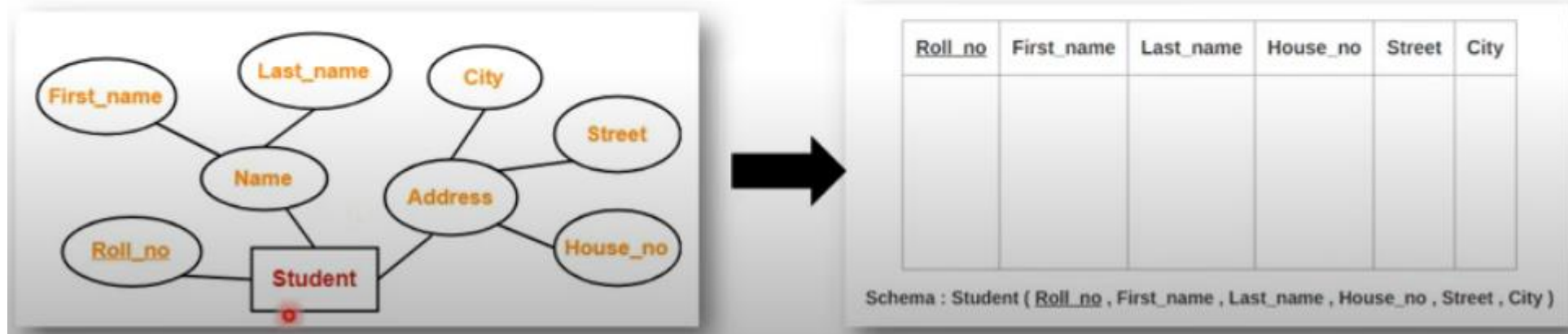


<u>Roll_no</u>	Name	Sex

Schema : Student (Roll_no , Name , Sex)

➤ Rule 2: For Strong Entity Set With Composite Attributes

- A strong entity set with any number of composite attributes will require only one table in relational model.
- While conversion, simple attributes of the composite attributes are taken into account and not the composite attribute itself.



➤ **Rule 3: For Strong Entity Set With Multi Valued Attributes**

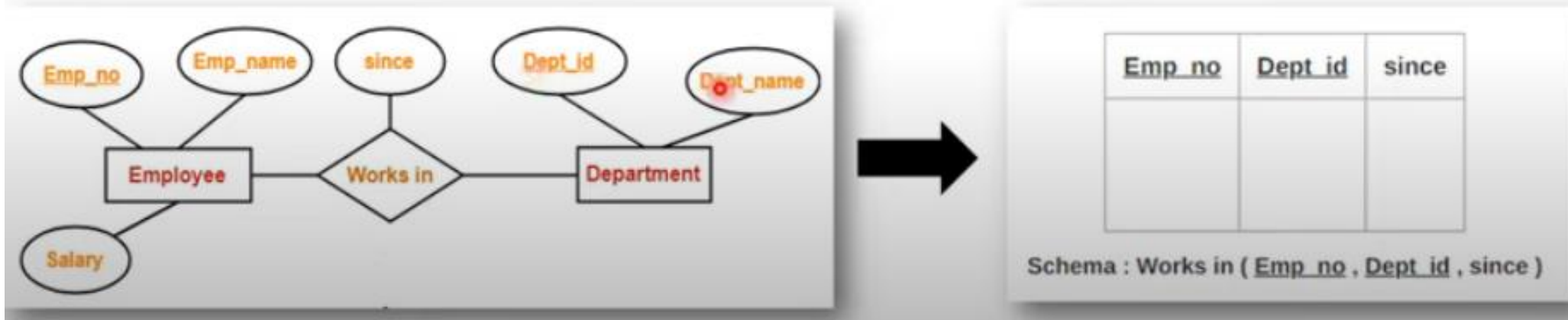
- A strong entity set with any number of multi valued attributes will require two tables in relational model.
- One table will contain all the simple attributes with the primary key.
- Other table will contain the primary key and all the multi valued attributes.



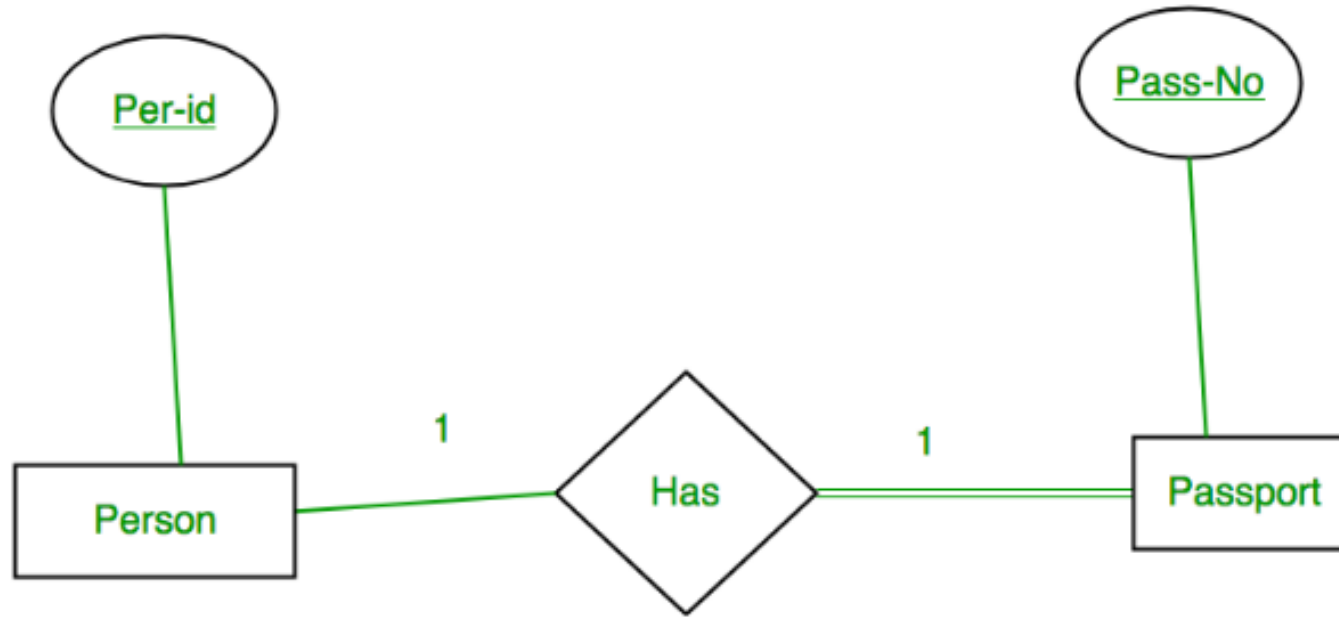
➤ Rule 4: Translating Relationship Set into a Table

Attributes of the table are-

- Primary key attributes of the participating entity sets
- Its own descriptive attributes if any.
- Set of non-descriptive attributes will be the primary key.



Case 1: Binary Relationship with 1:1 cardinality with total participation of an entity



A person has 0 or 1 passport number and Passport is always owned by 1 person. So it is 1:1 cardinality with full participation constraint from Passport.

First Convert each entity and relationship to tables. Person table corresponds to Person Entity with key as Per-Id. Similarly Passport table corresponds to Passport Entity with key as Pass-No. Has Table represents relationship between Person and Passport (Which person has which passport). So it will take attribute Per-Id from Person and Pass-No from Passport.

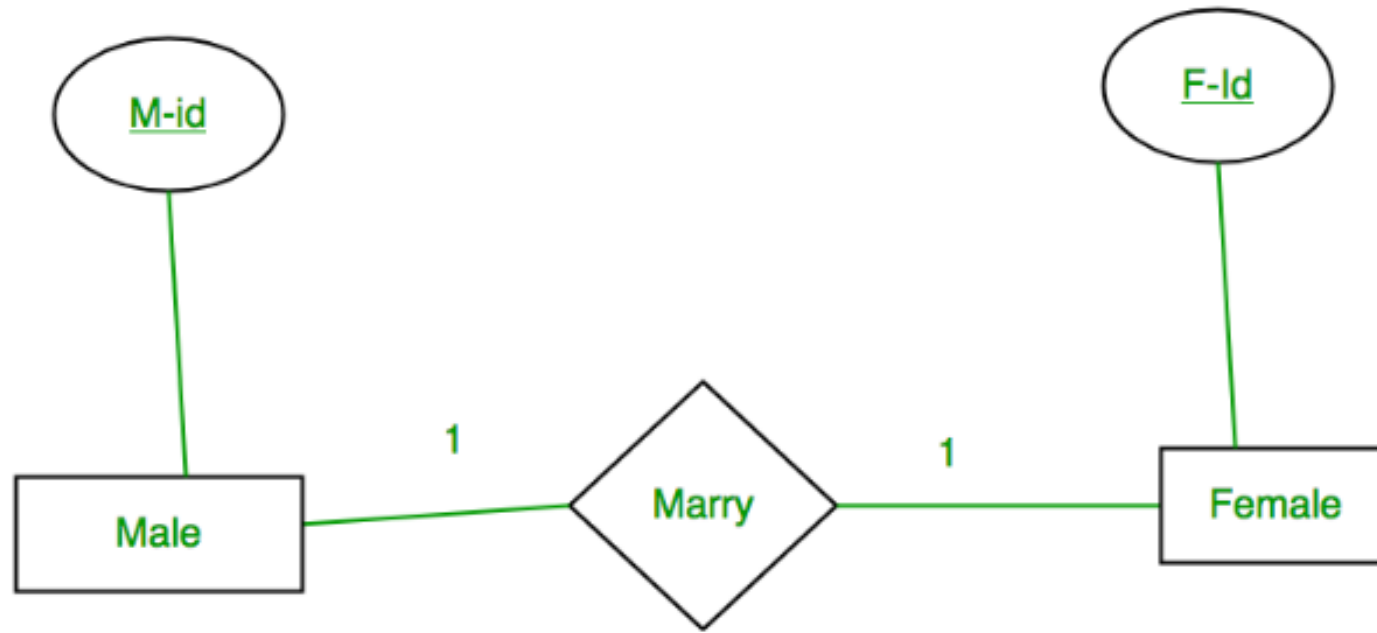
Person			Has			Passport	
<u>Per-Id</u>	Other Person Attribute		<u>Per-Id</u>	Pass-No		<u>Pass-No</u>	Other PassportAttribute
PR1	-		PR1	PS1		PS1	-
PR2	-		PR2	PS2		PS2	-
PR3	-						

Table 1

As we can see from Table 1, each Per-Id and Pass-No has only one entry in Has Table. So we can merge all three tables into 1 with attributes shown in Table 2. Each Per-Id will be unique and not null. So it will be the key. Pass-No can't be key because for some person, it can be NULL.

<u>Per-Id</u>	Other Person Attribute	Pass-No	Other PassportAttribute
---------------	------------------------	---------	-------------------------

Case 2: Binary Relationship with 1:1 cardinality and partial participation of both entities



A male marries 0 or 1 female and vice versa as well. So it is 1:1 cardinality with partial participation constraint from both. First Convert each entity and relationship to tables. Male table corresponds to Male Entity with key as M-Id. Similarly Female table corresponds to Female Entity with key as F-Id. Marry Table represents relationship between Male and Female (Which Male marries which female). So it will take attribute M-Id from Male and F-Id from Female.

Male			Marry			Female	
<u>M-Id</u>	Other Male Attribute		<u>M-Id</u>	F-Id		<u>F-Id</u>	Other FemaleAttribute
M1	-		M1	F2		F1	-
M2	-		M2	F1		F2	-
M3	-					F3	-

Table 3

As we can see from Table 3, some males and some females do not marry. If we merge 3 tables into 1, for some M-Id, F-Id will be NULL. So there is no attribute which is always not NULL. So we can't merge all three tables into 1. We can convert into 2 tables. In table 4, M-Id who are married will have F-Id associated. For others, it will be NULL. Table 5 will have information of all females. Primary Keys have been underlined.

<u>M-Id</u>	Other Male Attribute	F-Id
-------------	----------------------	------

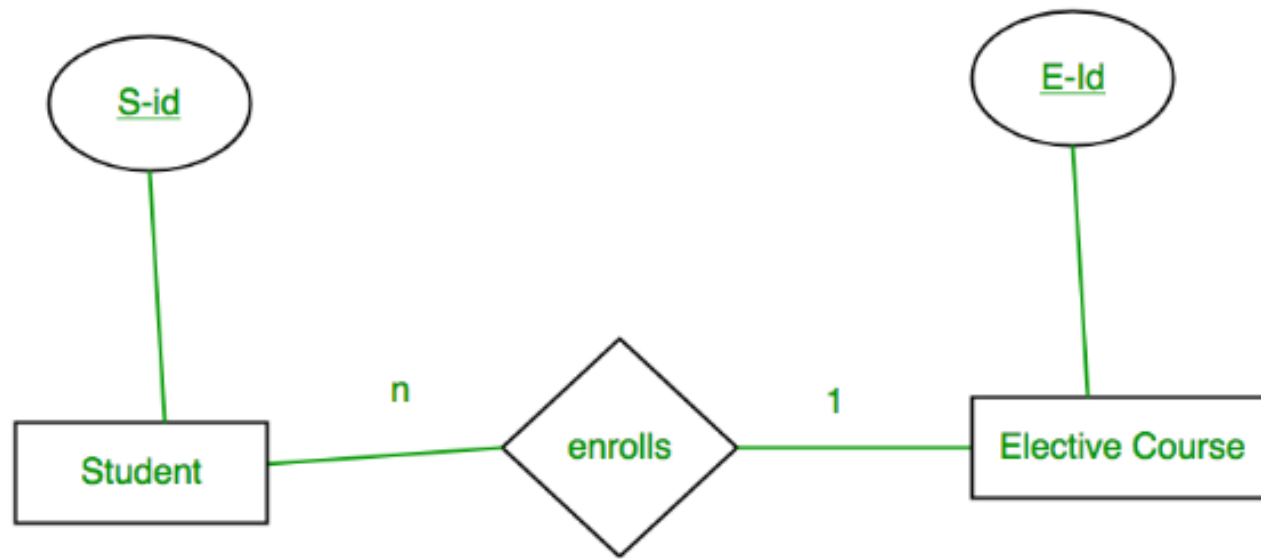
Table 4

<u>F-Id</u>	Other FemaleAttribute
-------------	-----------------------

Table 5

Note: Binary relationship with 1:1 cardinality will have 2 table if partial participation of both entities in the relationship. If atleast 1 entity has total participation, number of tables required will be 1.

Case 3: Binary Relationship with n: 1 cardinality



In this scenario, every student can enroll only in one elective course but for an elective course there can be more than one student. First Convert each entity and relationship to tables. Student table corresponds to Student Entity with key as S-Id. Similarly Elective_Course table corresponds to Elective_Course Entity with key as E-Id. Enrolls Table represents relationship between Student and Elective_Course (Which student enrolls in which course). So it will take attribute S-Id from Student and E-Id from Elective_Course.

Student		Enrolls		Elective_Course	
<u>S-Id</u>	Other Student Attribute	<u>S-Id</u>	E-Id	<u>E-Id</u>	Other Elective CourseAttribute
S1	-	S1	E1	E1	-
S2	-	S2	E2	E2	-
S3	-	S3	E1	E3	-
S4	-	S4	E1		

Table 6

As we can see from Table 6, S-Id is not repeating in Enrolls Table. So it can be considered as a key of Enrolls table. Both Student and Enrolls Table's key is same. We can merge it as a single table. The resultant tables are shown in Table 7 and Table 8. Primary Keys have been underlined.

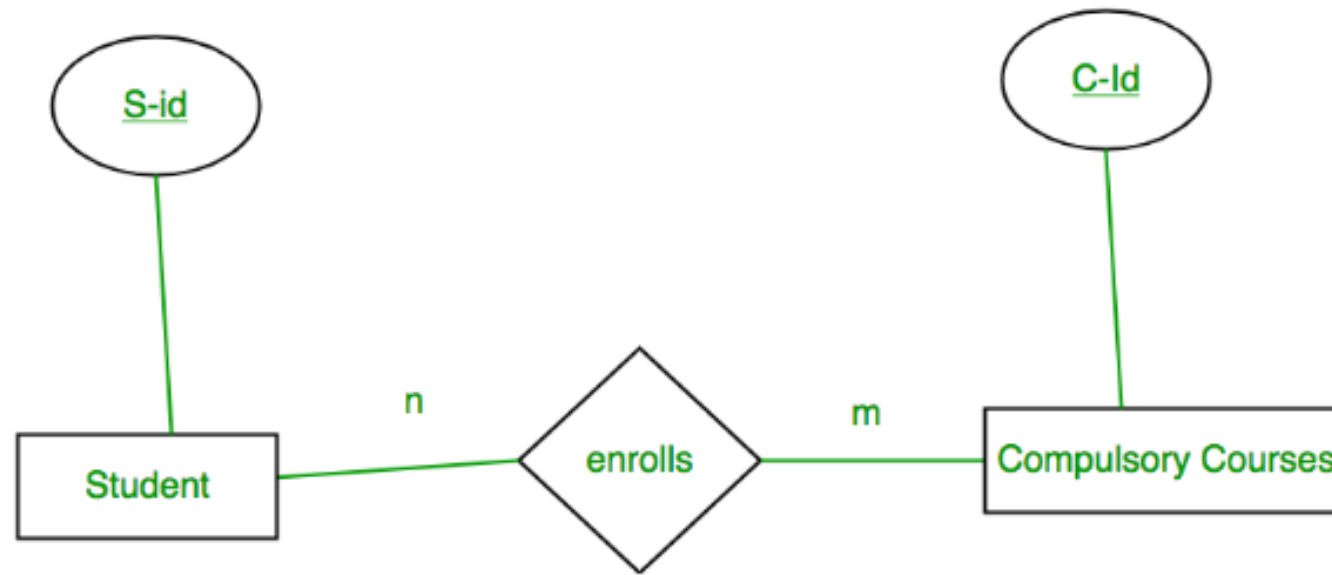
<u>S-Id</u>	Other Student Attribute	E-Id
-------------	-------------------------	------

Table 7

<u>E-Id</u>	Other Elective CourseAttribute
-------------	--------------------------------

Table 8

Case 4: Binary Relationship with m: n cardinality



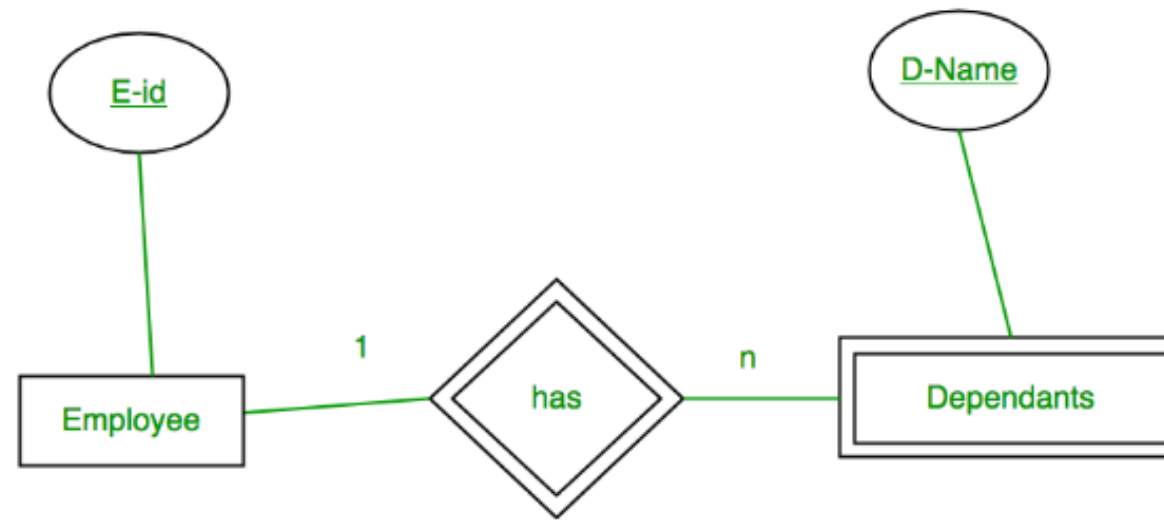
In this scenario, every student can enroll in more than 1 compulsory course and for a compulsory course there can be more than 1 student. First Convert each entity and relationship to tables. Student table corresponds to Student Entity with key as S-Id. Similarly Compulsory_Courses table corresponds to Compulsory Courses Entity with key as C-Id. Enrolls Table represents relationship between Student and Compulsory_Courses (Which student enrolls in which course). So it will take attribute S-Id from Person and C-Id from Compulsory_Courses.

Student		Enrolls		Compulsory_Courses	
<u>S-Id</u>	Other Student Attribute	<u>S-Id</u>	<u>C-Id</u>	<u>C-Id</u>	Other Compulsory CourseAttribute
S1	-	S1	C1	C1	-
S2	-	S1	C2	C2	-
S3	-	S3	C1	C3	-
S4	-	S4	C3	C4	-
		S4	C2		
		S3	C3		

Table 9

As we can see from Table 9, S-Id and C-Id both are repeating in Enrolls Table. But its combination is unique; so it can be considered as a key of Enrolls table. All tables' keys are different, these can't be merged. Primary Keys of all tables have been underlined.

Case 5: Binary Relationship with weak entity



In this scenario, an employee can have many dependents and one dependent can depend on one employee. A dependent does not have any existence without an employee (e.g; you as a child can be dependent of your father in his company). So it will be a weak entity and its participation will always be total. Weak Entity does not have key of its own. So its key will be combination of key of its identifying entity (E-Id of Employee in this case) and its partial key (D-Name).

First Convert each entity and relationship to tables. Employee table corresponds to Employee Entity with key as E-Id. Similarly Dependents table corresponds to Dependent Entity with key as D-Name and E-Id. Has Table represents relationship between Employee and Dependents (Which employee has which dependents). So it will take attribute E-Id from Employee and D-Name from Dependents.

Employee		Has		Dependents		
<u>E-Id</u>	Other Employee Attribute	<u>E-Id</u>	<u>D-Name</u>	<u>D-Name</u>	<u>E-Id</u>	Other DependentsAttribute
E1	-	E1	RAM	RAM	E1	-
E2	-	E1	SRINI	SRINI	E1	-
E3	-	E2	RAM	RAM	E2	-
		E3	ASHISH	ASHISH	E3	-

Table 10

As we can see from Table 10, E-Id, D-Name is key for **Has** as well as Dependents Table. So we can merge these two into 1. So the resultant tables are shown in Tables 11 and 12. Primary Keys of all tables have been underlined.

<u>E-Id</u>	Other Employee Attribute
-------------	--------------------------

Table 11

<u>D-Name</u>	<u>E-Id</u>	Other DependentsAttribute
---------------	-------------	---------------------------